



RITAL

Sentiment classification
Speakers classification

Paul Béglin
Ekaterina Bogush

29 March 2026

Contents

1	Introduction	2
2	EDA	2
2.1	Preprocessing	2
2.2	EDA	2
2.3	LDA	5
3	Experimental Details	8
3.1	Train/Test Separation	8
3.2	Metrics	8
3.3	Hardware	9
4	Classification	9
4.1	Simple models	9
4.2	Vectorization techniques	9
5	Movies classification	10
5.1	BERT transformer	11
5.2	DeBERTa	12
6	Presidents classification	12
6.1	Baseline models	12
6.2	BERT	13
6.3	BiLSTM on speech-level	17
6.4	Model Comparison and Ensembling	19
7	Conclusion	20
	Appendix	21

1 Introduction

In this project, our goal is to explore different binary classification techniques on two text datasets: one composed of movie reviews and another composed of sentences from presidential speeches. We investigate a range of vectorization approaches, from simpler methods such as BoW and TF-IDF to state-of-the-art transformer embeddings. Similarly, we evaluate models spanning classical classifiers (e.g., logistic regression) to fine-tuned transformers and BiLSTM neural networks.

2 EDA

2.1 Preprocessing

2.1.1 For TF-IDF, BoW

For each dataset our preprocessing pipeline for classic vectorization techniques like BoW and for preliminary analysis of datasets consists of several common steps:

1. URLs, phone numbers, and email addresses are removed using regular expressions, as they typically do not contribute meaningful information for the task and are reviews specific;
2. Digits are removed since numerical values are assumed to provide limited semantic information (the point discussed on lecture, that it is the case even for sentiment classification) for classification, and we suppose that textual vocabulary carries much more information for the models;
3. Punctuation to simplify the feature space;
4. Normalization Form C (NFC) text normalization is applied to remove non-standard characters while preserving accents, which is important for French text;
5. Stopwords are removed using lists from the `nltk` library, extended with additional common words (e.g., `yeah`, `oh`);
6. Lemmatization is performed using `spacy` to reduce words to their base forms. Stemming is not used, as lemmatization simplifies interpretability of datasets and improves compatibility with pretrained word embeddings like Word2Vec.

2.1.2 For transformers

In contrast to preprocessing typically used for classical vectorization methods, we applied minimal (or no) preprocessing to the datasets. This is because transformer-based models such as BERT can directly process raw text. For the movie dataset, phone numbers, emails, and URLs were replaced with placeholders (PHONE, EMAIL, URL), while no preprocessing was applied to the presidents dataset.

2.2 EDA

The movie dataset contains in total 2 000 movies reviews and classes (0 for negative, 1 for positive) are perfectly balanced ; presidents datasets contains in total 50 000 phrases and highly imbalanced with class ratio close to 7:1 (-1 for Mitterrand, 1 for Chirac).

Figure 2 shows the distribution of document lengths (in number of words). This is particularly important when using transformers, as their context window size is limited.

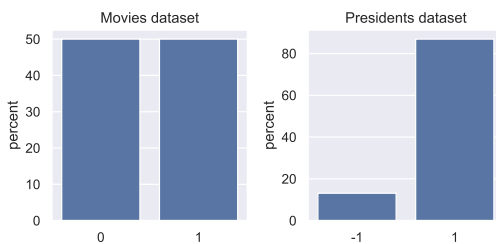


Figure 1: Class distribution across the different datasets

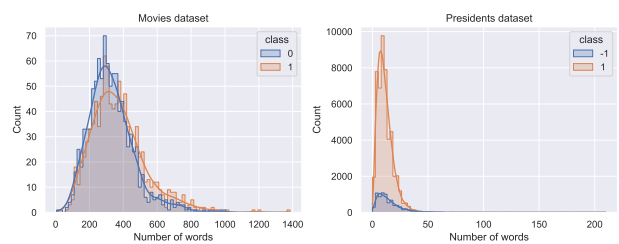


Figure 2: Distribution of document lengths (in number of words)

Of note, after a closer examination of the presidential phrases, we concluded that the examples are not independent. It is likely that entire paragraphs from each president’s speeches were segmented into individual phrases, introducing sequential dependencies between them. This dependency must be taken into account, as the examples are not i.i.d., which is a common assumption for standard models such as SVM, logistic regression, and conventional oversampling techniques.

Moreover, the length (in terms of consecutive phrases) of speeches varies across presidents (Figure 3). For Mitterrand, the mean number of consecutive phrases is 18.8, with a maximum of 39. For Chirac, these values are 124.4 and 952, respectively. The minimum length is 3 phrases for both presidents.

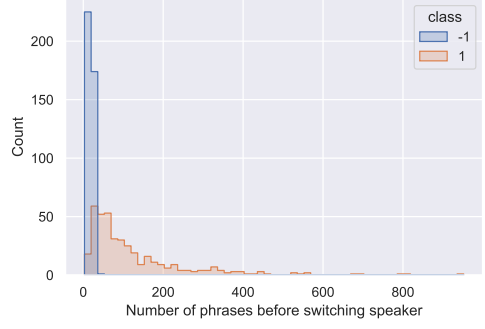


Figure 3: Distribution of the number of consecutive phrases per president

2.2.1 Building vocabulary

To limit the vocabulary size, we set a minimum document frequency threshold of 5, meaning that each word must appear in at least five documents. This helps filter out rare words, such as typos or terms specific to a single movie or speech.

For the maximum document frequency threshold, which removes words appearing in more than $x\%$ of documents), we examined word clouds for each class separately (weighted by word frequency). When the most frequent words appeared similar across both classes, we reduced the threshold. For example, with a maximum document frequency of 0.8, Figure 4 shows the corresponding word clouds.



(a) Movies dataset



(b) Presidents dataset

Figure 4: Word clouds with a maximum document frequency of 0.8

By experimenting with different thresholds, we settled on a maximum document frequency of 0.5. Noticeable differences appear in the most frequent words for the movies dataset depending on this threshold. For the presidents dataset, the maximum document frequency of words is 0.15 due to the short length of the documents. As a result, the maximum document frequency threshold has a no impact on this dataset and we won’t reduce it further.



(a) Movies dataset



(b) Presidents dataset

Figure 5: Word clouds with a maximum document frequency of 0.5

2.2.2 Zipf law

Figure 7 illustrates the theoretical Zipf distribution compared to the empirical distribution for each dataset (no minimum or maximum document frequency filtering was applied before plotting). For lower ranks, the empirical and theoretical curves closely match. For higher ranks, the gap between them increases, which is commonly observed in real-world datasets.



(a) Movies dataset (2-grams)



(b) Presidents dataset (2-grams)



(c) Movies dataset (3-grams)



(d) Presidents dataset (3-grams)

Figure 6: Word clouds for 2-grams and 3-grams with a maximum document frequency threshold of 0.5

This discrepancy can be explained by the presence of typos and rare words, as well as by the fact that Zipf's law assumes an infinite corpus, whereas real datasets are finite and do not contain enough observations for low-frequency terms. Additionally, for higher ranks we observe a step-like pattern, indicating that many rare words occur with the same low frequency (typically once or twice).

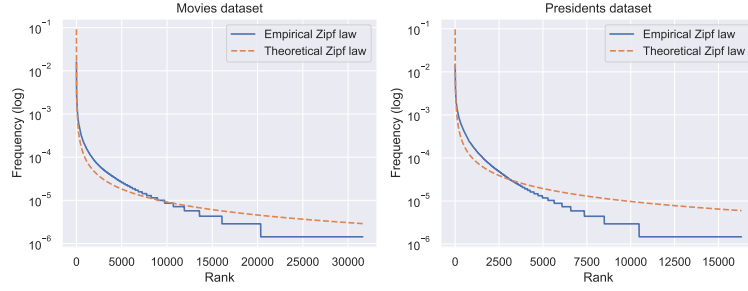


Figure 7: Comparison of theoretical and empirical Zipf distributions

2.2.3 Log-odds

Figure 8 illustrates the most discriminative words (after applying the document frequency threshold) based on the log-odds ratio for each class in each dataset. For a given word w , we define its empirical class probability as:

$$p_c(w) = \frac{N_c(w)}{N_c}$$

$$\text{LogOddsRatio}(w) = \log \left(\frac{\frac{p_0(w)}{1-p_0(w)}}{\frac{p_1(w)}{1-p_1(w)}} \right)$$

where:

- $N_c(w)$ is the number of documents in class c that contain the word w ,
- N_c is the total number of documents in class c .

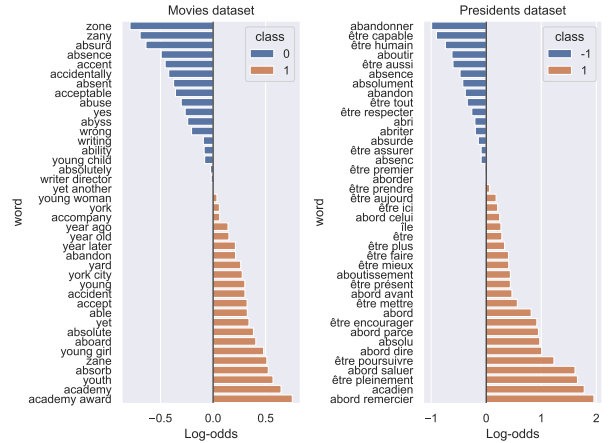


Figure 8: Most discriminative words per class based on log-odds ratios

The log-odds ratio ranges from $-\infty$ to $+\infty$. A positive value indicates that the word is more strongly associated with class y_0 , while a negative value indicates a stronger association with class y_1 .

2.3 LDA

To better understand the documents in each dataset, we used a topic modeling technique called LDA, which assigns each document a probability distribution over latent topics.

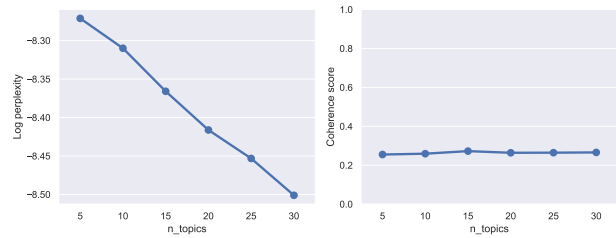
Note: LDA is used here only as part of the exploratory data analysis and for gaining intuition about the datasets; it is not used later for classification.

A common challenge with LDA is choosing the number of topics. To address this, we test different numbers of topics and evaluate them using log perplexity (unbounded, lower is better). Since perplexity tends to decrease as the number of topics increases, we also rely on the coherence score C_V (higher is better), which lies in $[0, 1]$. The coherence score measures how often words within the same topic co-occur in documents (using a sliding window over n-grams) and is designed to reflect how interpretable the topics are to humans.

For each dataset, we select the number of topics that provides a good trade-off between these metrics, and then inspect the intertopic distance map. We also perform UMAP using the Hellinger distance – a metric commonly used to compare probability distributions – and visualize documents colored by their dominant topic.

2.3.1 Movies

Figure 9 illustrates how log perplexity and coherence vary with the number of topics. As expected, log perplexity decreases as the number of topics increases. However, the coherence score remains relatively stable, around 0.3, which is on the lower side and typically indicates that the topics are not very interpretable.



To explore the topics more visually, we fix the number of topics to 10. Figure 10 shows the intertopic distance map. By inspecting the most relevant terms, we can identify topics associated with horror movies (topic 7), Disney-like movies (topic 8), and family/comedy movies (topic 1).

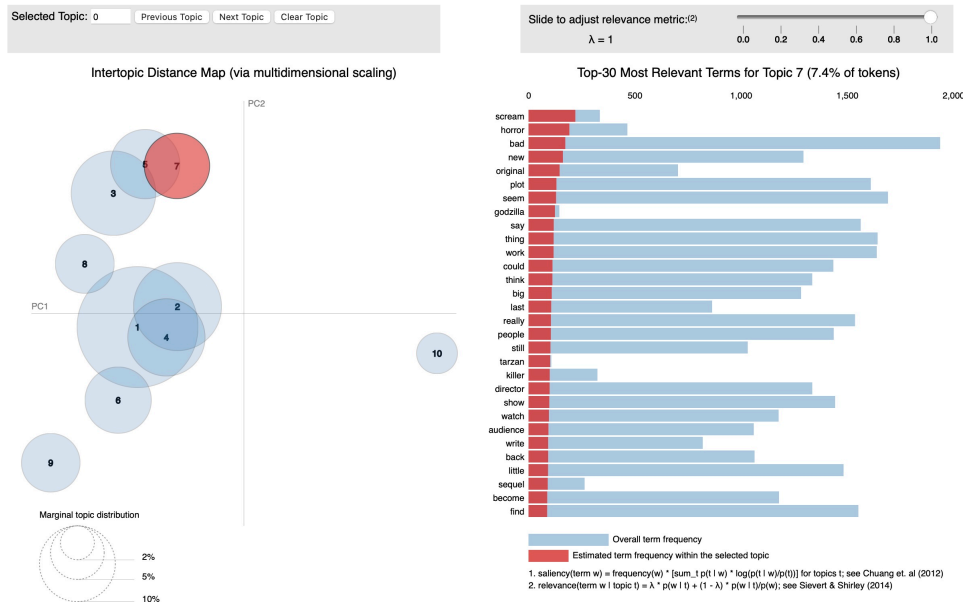


Figure 10: Intertopic distance map for the LDA model with 10 topics (movies)

Figure 11 shows an extract (first 100 rows) of the document-topic distribution matrix. In most cases, a single topic clearly dominates each document.

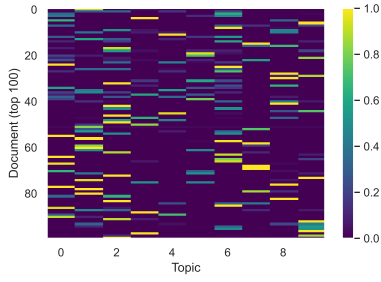


Figure 11: Excerpt of the document-topic distribution matrix (movies)

As a final visualization, we apply UMAP dimensionality reduction (Figure 12, left) to the document-topic matrix, with documents colored by their dominant topic. Some topics are clearly separated (e.g., topics 3, 5, and 7), while others appear more mixed (e.g., topic 9). This likely reflects overlap between topics or documents covering multiple themes.

In the right part of Figure 12, documents are colored by their class. There is no clear separation between classes in this representation, suggesting that the topic space alone does not strongly discriminate between them.

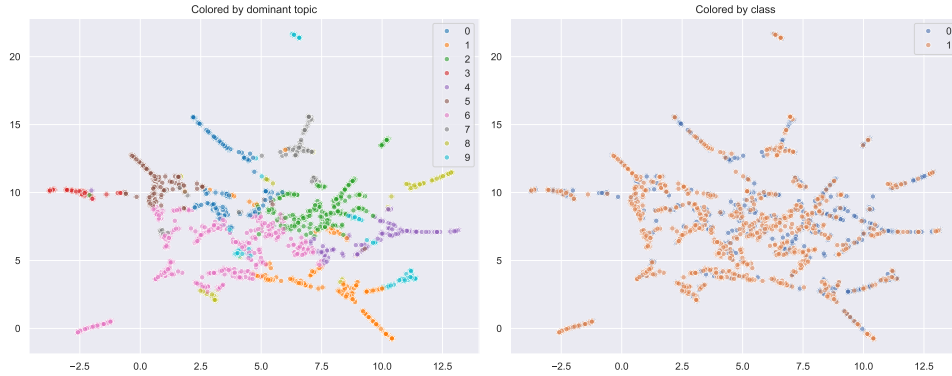


Figure 12: UMAP projection of the document-topic matrix (movies), colored by dominant topic (left) and class (right)

2.3.2 Presidents

The main challenge with the presidents dataset is that each document typically consists of a single phrase. However, LDA assumes that each document is a mixture of multiple topics, which is rarely the case when documents are that short. We first attempted to run LDA directly on individual phrases (e.g., with 15 topics), but as expected, the model struggled to identify coherent topics. By inspecting the intertopic distance map (Figure 13), we observed that the most relevant words in each topic were mostly common verbs such as *pouvoir* and *avoir*, making the topics difficult to interpret.

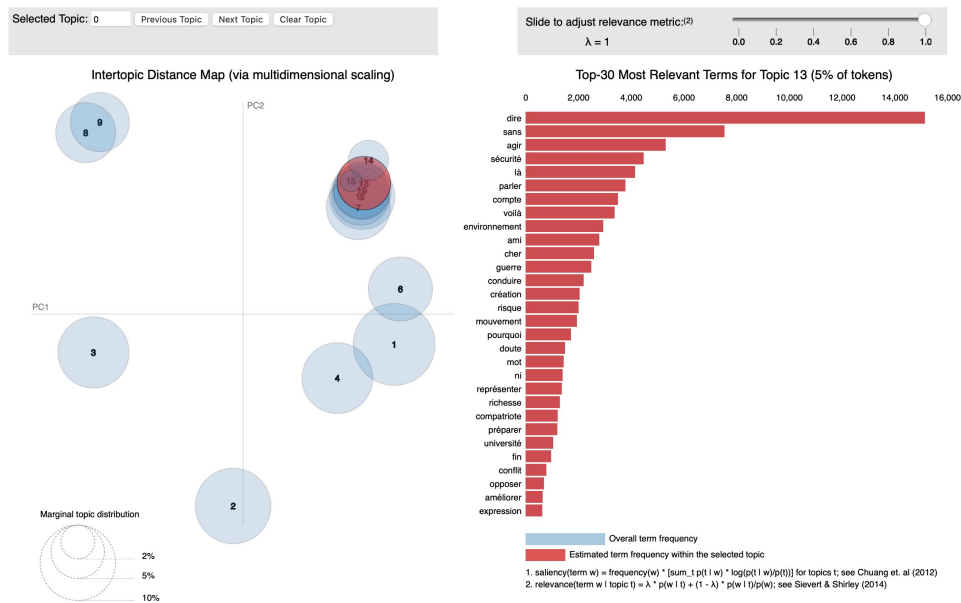


Figure 13: Intertopic distance map (15 topics) for LDA on individual phrases (presidents)

To address this issue, we concatenated consecutive phrases from the same president to form longer documents. We then evaluated LDA for different numbers of topics (Figure 14) and selected 15 topics for further analysis.

Although the coherence score remains relatively low, the resulting topics are much more interpretable. We can identify (Figure 15), for example, topics related to culture and art (topic 7), national defense (topic 12), cooperation with overseas countries (topic 11), health (topic 9), and social services (topic 1).

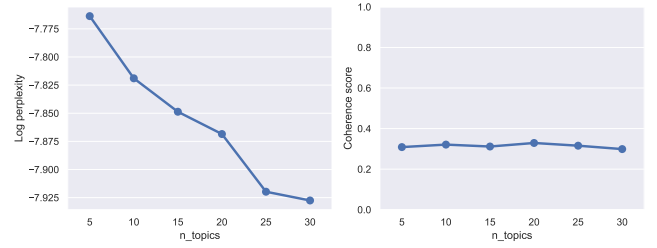


Figure 14: LDA evaluation metrics (log perplexity and coherence) versus number of topics for the presidents dataset

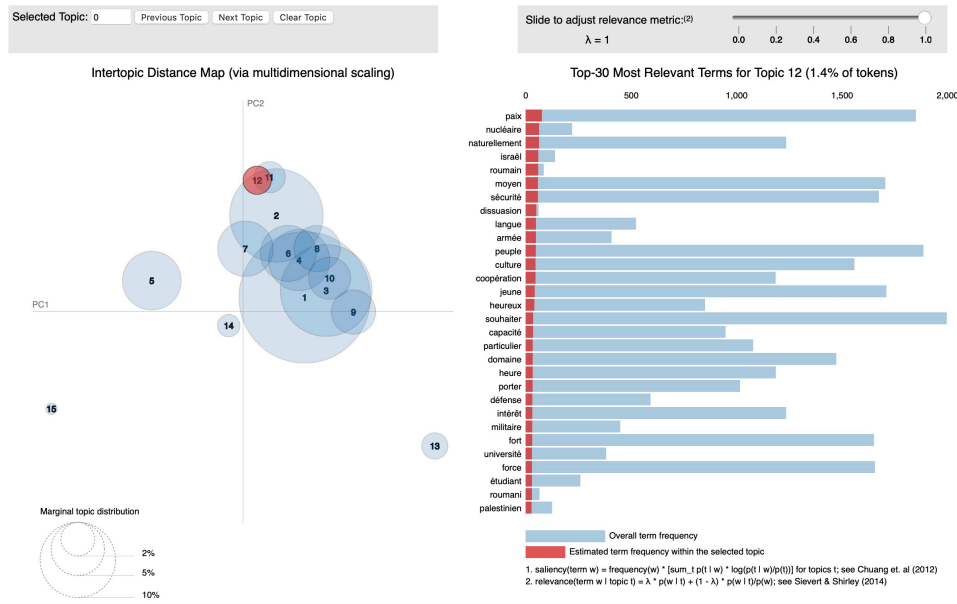


Figure 15: Intertopic distance map (15 topics) for LDA on concatenated phrases (presidents)

Figure 16 shows an extract (first 100 rows) of the document-topic distribution matrix. Compared to the movies dataset, documents here tend to be mixtures of several topics rather than being dominated by a single one.

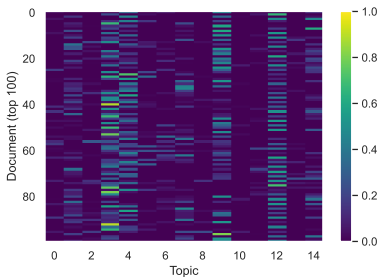


Figure 16: Excerpt of the document-topic distribution matrix (presidents dataset)

Figure 17 presents the UMAP projection of the document-topic matrix, with documents colored by their dominant topic. Documents with the same dominant topic tend to form clusters, although some overlap remains. This is consistent with the fact that each document is a mixture of multiple topics, as also seen in Figure 16.

On the right of Figure 17, documents are colored by class. The representation shows a clearer separation between presidents, with limited overlap. This suggests that different presidents tend to emphasize different topics in their speeches. For instance, Mitterrand appears to focus more on public rights (topic 12 in the UMAP), while Chirac places more emphasis on topics related to emergency services such as fire brigades and the gendarmerie (topic 3 in the UMAP).

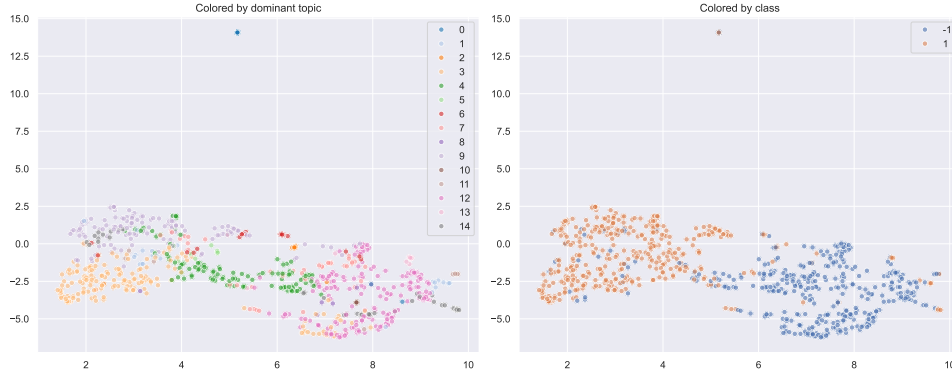


Figure 17: UMAP projection of the document-topic matrix for the presidents dataset

3 Experimental Details

3.1 Train/Test Separation

3.1.1 Movies

Movie reviews are independent from one another. Therefore, we adopt a standard shuffled train/test split, allocating 80% of the data for training and 20% for testing. The same approach is applied to 5-fold cross-validation.

3.1.2 Presidents

Given the presence of sequential dependencies, random shuffling cannot be used to divide the data into train and test subsets, as this would disrupt the temporal structure (post-processing techniques such as smoothing ineffective). Consequently, the train/test split is performed using a chronological cutoff: the first 80% of the dataset is assigned to the training set, and the remaining 20% to the test set. The class distribution in each subset is approximately preserved. For cross-validation, in order to preserve the class imbalance, the phrases of each speaker are assigned to a fixed number of blocks and then interleaved, while maintaining their original order. This may result in phrases from the same presidential speech being placed into different blocks, which we will tolerate.

3.1.3 Leaderboard

The models that we submit for the leaderboard are trained on the full train dataset. They are highlighted in yellow and enumerated with (.).

3.2 Metrics

To evaluate the performance of our models, we rely on several metrics:

- Accuracy, which measures the proportion of correct predictions made by the model.
- Precision (binary averaging), which measures the proportion of predicted positives that are actually positive.
- Recall (binary averaging), which measures the proportion of actual positives that the model correctly identifies.
- F1-score, which captures the balance between precision and recall.

Accuracy, precision, recall, and F1-score are defined with respect to a specific decision threshold.

- ROC AUC, which evaluates the model’s ability to rank positive instances higher than negative ones, independently of a specific decision threshold.
- PR AUC, which evaluates the trade-off between precision and recall across thresholds, and is particularly useful for imbalanced dataset.

We compute these metrics for both datasets, while reporting PR AUC only for the presidents dataset due to class imbalance. For the movies dataset, we consider positive reviews (class 1) as the positive class. For the presidents dataset, we treat Mitterrand (class -1, minority class) as the positive class in order to better evaluate the model’s ability to identify the minority class, rather than focusing on the majority class, which would lead to misleadingly high scores.

3.3 Hardware

For heavy, resource-demanding models such as transformers and neural networks, we used an NVIDIA all-in-one DGX Spark machine. The main challenge was properly installing PyTorch with CUDA 13.0 support while fully relying on the uv package manager. The available wheels required Python 3.14, likely due to the end of GIL, whereas other packages such as `gensim` or `spaCy` required Python versions below 3.14. As a result, the project uses 2 virtual environments: one with Python 3.13 and another with Python 3.14.

4 Classification

First, we outline the common elements of the pipelines for both the movies and presidents datasets.

4.1 Simple models

For baseline models, we focus on common approaches: logistic regression, linear SVM (without a kernel), and naive Bayes. Logistic regression and linear SVM are linear classifiers that learn a separating hyperplane between classes, while Naive Bayes is a probabilistic model that assumes a multinomial feature distribution and is therefore used only with BoW vectorization. In the presidents dataset, because we explore probability-smoothing strategies, we exclude linear SVM since it does not natively provide probability estimates.

4.2 Vectorization techniques

4.2.1 Classic vectorization techniques

To be able to use any machine learning models, each text must be mapped into vectors of the same dimension. First, we explore the most basic vectorization techniques like Bag-of-Words and TF-IDF with different tokenization units : character 1-gram, characters 4-grams, words, words 1-2 grams, words 1-3 grams. As for the dimensionality, we limit number of features to 10 000 (10 000 most common tokenization units) and also, since TF-IDF and BoW matrices are very sparse, we use LSA dimensionality reduction technique to compare if the performance improves over taking 10 000 most frequent tokenization units. For LSA we fix the number of components to 1000 with the goal to explain at least 80% of variance and reduce by at least 6 times .

Note : BoW is L1-normalized per row to have a probability distribution and to be able to use models like Logistic regression or SVM which expect vectors on the same scale. We also perform L2-normalization to the matrix returned by the LSA technique since features can be arbitraly large.

4.2.2 Static word embeddings

We also test vectorization using static word embeddings models, like pretrained Word2Vec and FastText models.

Since Word2Vec model work on word units and FastText on subwords units, in order to transform an entire text into a single embedding, we use different so-called pooling techniques:

- Mean pooling: average of all token embeddings.
- Max pooling: element-wise maximum over token embeddings.
- Mean-max pooling: concatenation of mean and max pooled vectors (dimension is doubled).
- TF-IDF pooling: weighted mean of token embeddings, where each token embedding is weighted by its TF-IDF score.

$$\mathbf{d} = \frac{\sum_i w_i \mathbf{e}_i}{\sum_i w_i}$$

- w_i is the TF-IDF weight of token i
- $\mathbf{e}_i$ is its embedding vector

- SIF¹ pooling: weighted mean of token embeddings, where each token embedding is weighted by its inversed frequency.

$$w_i = \frac{a}{a + p(w_i)},$$

$$\mathbf{d} = \frac{\sum_i w_i \mathbf{e}_i}{\sum_i w_i},$$

$$\mathbf{d} \leftarrow \mathbf{d} - \langle \mathbf{d}, \mathbf{u} \rangle \mathbf{u}$$

- $p(w_i)$ is the frequency (probability) of token i in the corpus
- a is a smoothing constant, 0.001 in our case
- $\mathbf{u}$ is the first principal component computed over document embeddings (used to remove noise and the information shared across all documents)

4.2.3 BERT Transformers

Finally, we explore transformers from HuggingFace based on BERT architecture. We explore both getting embeddings from the transformer and passing them to simpler models and then finetuning both embeddings layers and classification head of the selected transformers.

5 Movies classification

This section describes the models specific to the movies dataset and the results obtained.

5.0.1 Simple models results

Figure 18 shows the performance (5-fold cross-validation) of Logistic Regression, Linear SVM, and Naive Bayes classifiers. Similar trends are observed on a train/test split, with comparable values across all metrics. For the more complex experiments presented later, we therefore report results mainly using the train/test split evaluation.

For static word embeddings, we used pretrained models trained on English corpora: the Word2Vec model *word2vec-google-news-300* available in *gensim*, and the FastText model *cc.en.300.bin*.

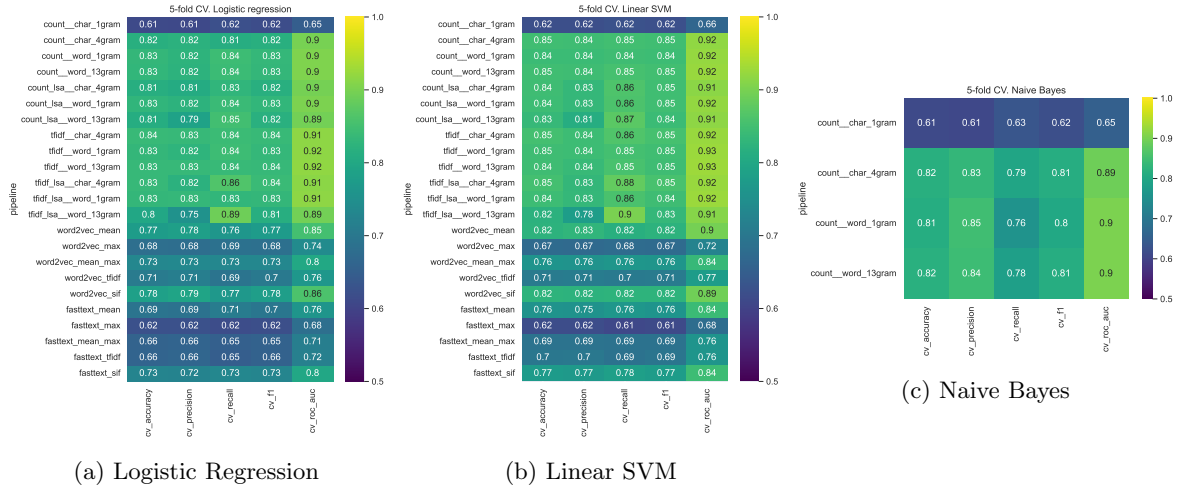


Figure 18: Baseline models performance across vectorization techniques (5-fold cross-validation)

Surprisingly, simple vectorization techniques such as TF-IDF and BoW outperform static word embeddings. Among embedding-based approaches, SIF pooling performs the best, clearly outperforming the other pooling strategies. Logistic Regression, Linear SVM, and Naive Bayes show very similar overall performance, although Linear SVM tends to achieve slightly better scores (by about 0.02 points).

Interestingly, the simplest tokenization unit we experimented with – character-level representations – was mainly used to assess how much performance could be achieved with minimal input. Despite its simplicity and a

¹Smooth Inverse Frequency

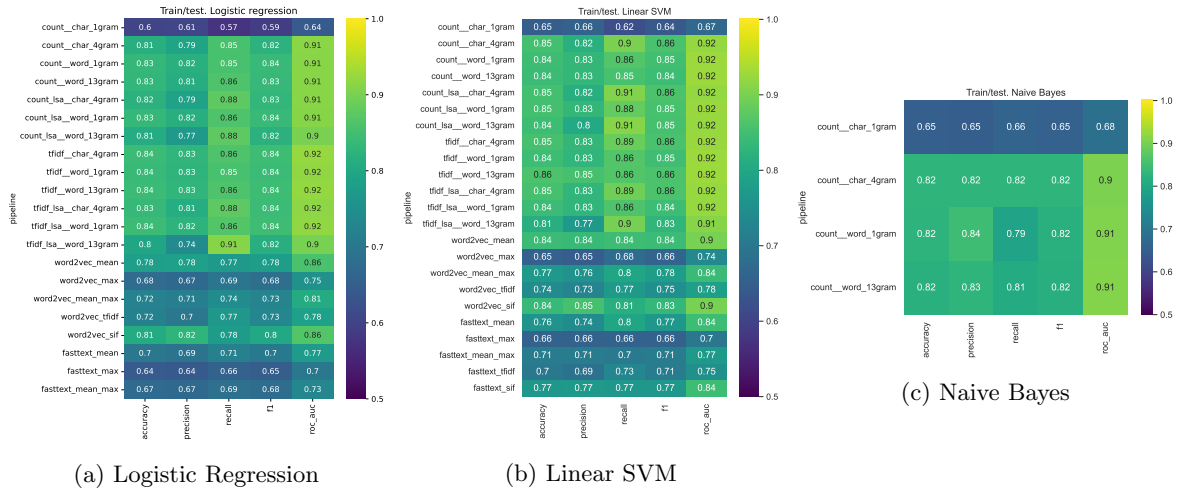


Figure 19: Baseline models performance across vectorization techniques (train/test split)

vocabulary of fewer than 40 characters, it still reaches around 0.6 across all metrics, which is relatively high given the simplicity of the representation.

5.1 BERT transformer

For the English dataset, we selected the transformer BERT-base-uncased. We preferred the uncased variant because the training data is already lowercased by default, whereas this is not the case for the true test dataset.

5.1.1 CLS token embedding

Our first approach was to extract semantically rich embeddings from the CLS token, which is intended to represent the whole input sequence, and use them as features for simpler classifiers such as Logistic Regression or Linear SVM. Since the maximum context length is 512 tokens, and more than half of the reviews exceed this length (Section 2.2), we split texts into chunks of 512 tokens using an overlapping window (stride of 128 tokens). As a result, each document is represented by multiple embeddings.

We assume that each chunk captures a local sentiment, since a single review may contain both positive and negative aspects. We train a Logistic Regression model on chunk embeddings, then aggregate predictions at the document level by averaging and max pooling the predicted probabilities. We apply the same approach for SVM, using either mean pooling of decision scores or majority voting.

Model	Accuracy	Precision	Recall	F1-score	ROC AUC
Logistic Regression, mean pooling (4)	0.89	0.85	0.93	0.89	0.94
Logistic Regression, max pooling	0.80	0.73	0.95	0.83	0.92
SVM, mean pooling	0.87	0.85	0.90	0.87	0.94
SVM, majority vote	0.87	0.88	0.85	0.87	—

Table 1: Train/test split results for classifiers on CLS embeddings

5.1.2 Finetuning

Our next approach was to finetune BERT. We experimented with different input construction strategies. First, we combined the first 255 tokens (HEAD) and the last 254 tokens (TAIL), representing each text as [CLS] HEAD [SEP] TAIL [SEP]. We also tried using only the first 510 tokens, i.e., [CLS] HEAD [SEP].

We then experimented with a larger model, RoBERTa, which often outperforms BERT. We finetuned it using only the HEAD tokens, as this configuration performed better on the true test set. We first froze the embedding layers and trained only the classification head for 1 epoch. Then, we unfroze all layers and continued training for 5 epochs, allowing deeper layers to adapt gradually while avoiding large initial updates.

Model	Accuracy	Precision	Recall	F1-score	ROC AUC
BERT, HEAD only (3)	0.82	0.79	0.87	0.83	0.89
BERT, HEAD + TAIL (2)	0.90	0.87	0.93	0.90	0.96
RoBERTa, HEAD only (5)	0.89	0.83	0.97	0.90	0.95

Table 2: Train/test split results for finetuned transformer models

5.2 DeBERTa

For fun, we also experimented with a SOTA model, DeBERTa, which is finetuned on a large IMDB dataset (25k reviews) (submission 1). We later realized that the test dataset corresponds to the training split of the Stanford IMDB dataset used to train this model. As a result, the model benefits from data leakage, which explains the inflated leaderboard scores.

6 Presidents classification

Before evaluating our models we mapped Mitterand, minority class to 1 to consider it as positive (previously -1) and Chirac to 0.

To handle class imbalance, we considered 2 options: either modifying the loss function to penalize the minority class more heavily, or relying on oversampling. Since each speaker has their own speech duration, oversampling, if not carefully managed, could disrupt this structure. Therefore, we opted to use a weighted loss function instead.

6.1 Baseline models

For static word embeddings, we used only FastText (*cc.fr.300.bin*) since **gensim** does not provide a commonly distributed pretrained Word2Vec model for French corpora.

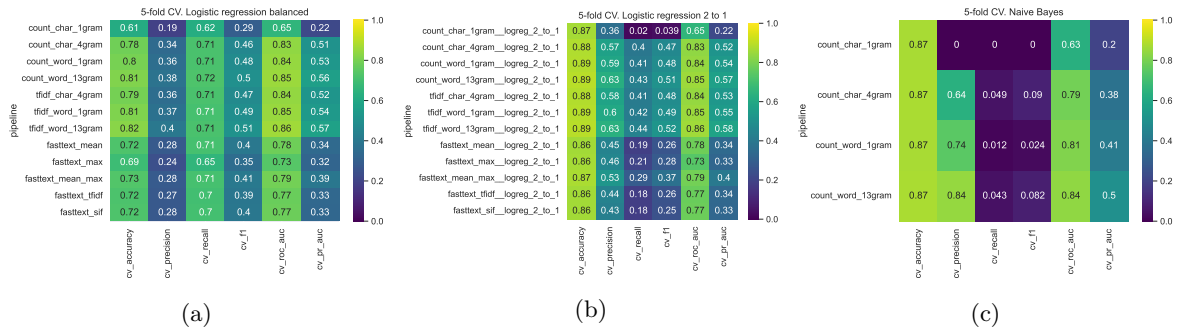


Figure 20: Baseline models performance across vectorization techniques (5-fold cross-validation)

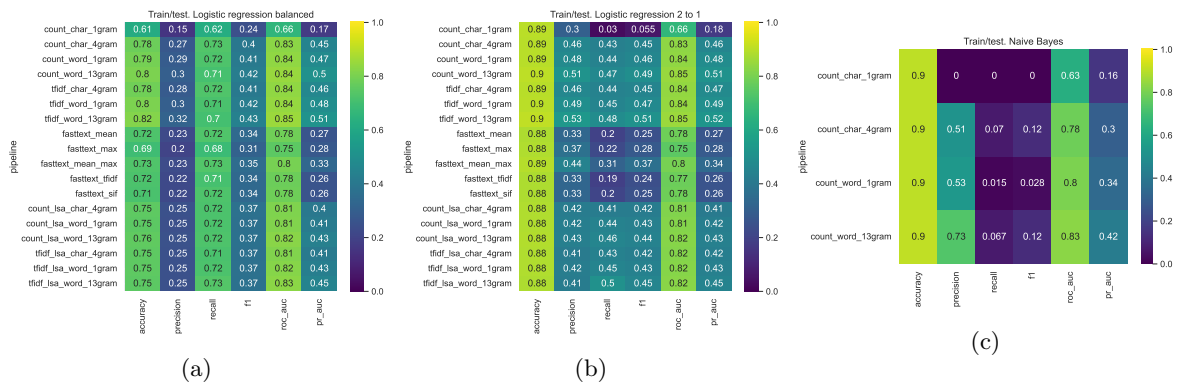


Figure 21: Baseline models performance across vectorization techniques (train/test split)

Results obtained with 5-fold cross-validation² are generally higher across metrics, whereas the train/test split

²We did not perform cross-validation on pipelines using LSA, as computing the LSA matrix was time-consuming.

tends to yield lower values across all metrics, which may be due to a more challenging split.

Naive Bayes showed poor performance on the Mitterrand class, consistently predicting Chirac only. This suggests that further hyperparameter tuning is required. With `class_weight=balanced`, logistic regression overcompensates by predicting Mitterrand too often, which significantly increases recall but lowers precision. A 2:1 penalty ratio provides a better balance, although both precision and recall remain relatively low. As observed on the movie dataset, TF-IDF and Bag-of-Words outperform FastText embeddings in this setting.

The high ROC AUC (> 0.8) can be misleading, as it is influenced by class imbalance favoring Chirac. In this context, false Mitterrand predictions are less critical than correctly identifying Chirac instances. However, the low PR AUC (< 0.6) better reflects the underlying issue: the model struggles to maintain good precision while achieving high recall.

6.2 BERT

For our transformer-based approach, we selected CamemBERT-large, whose greater capacity compared with the base version can better capture subtle semantic cues. Because the phrases are short and remain well below BERT’s 512-token context limit, we feed each full phrase directly into the model. Since phrases are fairly short and do not exceed the BERT context window size of 512 tokens we simply pass the whole phrase to the transformer.

6.2.1 CLS token embedding

For CLS token embeddings, we also attempted to make Logistic Regression sequence-aware through feature engineering (FE) by concatenating the following features to each phrase vector:

Feature block	Definition
Previous phrase embedding	$\mathbf{e}_{t-1}$
Current phrase embedding	$\mathbf{e}_t$
Next phrase embedding	$\mathbf{e}_{t+1}$
Cosine similarity (previous, current)	$\cos(\mathbf{e}_{t-1}, \mathbf{e}_t)$
Cosine similarity (current, next)	$\cos(\mathbf{e}_t, \mathbf{e}_{t+1})$
Current – previous embedding	$\mathbf{e}_t - \mathbf{e}_{t-1}$
Next – current embedding	$\mathbf{e}_{t+1} - \mathbf{e}_t$

Table 3: Sequence-aware feature engineering used with CLS embeddings for phrase-level classification

The final feature vector is the concatenation of all blocks in Table 3.

Because the first phrase has no previous context and the last phrase has no following context, and because logistic regression requires fixed-size feature vectors, we do not produce probabilities for these boundary phrases directly. Instead, we copy the probability predicted for the second phrase to the first phrase, and the probability predicted for the second-to-last phrase to the last phrase. This choice is justified by the observation that each speaker typically spans multiple consecutive phrases (Section 2.2 reports a minimum run length of 3 phrases).

Table 4 compares Logistic regression performance with and without feature engineering (FE). A 2:1 class weight indicates a weighted loss that assigns twice as much weight to Mitterrand as to Chirac; when no weight is reported, both classes are treated equally. Notably, weighted loss improves performance without FE, but degrades performance once FE is added.

Model	Accuracy	Precision	Recall	F1-score	ROC AUC	PR AUC	
Logistic regression	—	0.90	0.54	0.30	0.39	0.82	0.42
Logistic regression, weight 2:1	—	0.88	0.41	0.45	0.43	0.82	0.42
Logistic regression, weight 2:1, FE	—	0.89	0.46	0.63	0.53	0.89	0.56
Logistic regression, FE	—	0.91	0.54	0.54	0.54	0.89	0.57

Table 4: Logistic regression performance with and without FE (train/test split)

6.2.2 Smoothing

Our next goal was to eliminate low-confidence spikes occurring in the middle of a speech (e.g., probabilities slightly above 0.5 for Chirac and slightly below 0.5 for Mitterrand). We experimented with 3 approaches:

Gaussian smoothing, which consists of replacing each probability by a weighted average of its neighboring predictions, where closer points receive higher weights according to a Gaussian distribution. Figure 22 illustrates the effect of Gaussian smoothing (using the first 500 examples) for different values of the standard deviation σ , which controls the width of the Gaussian (denoted as `gauss_sigma` in the legend).

While Gaussian smoothing is effective at reducing spikes and correcting some Chirac predictions by lowering probabilities below the 0.5 threshold, it tends to overly smooth Mitterrand predictions. For instance, in the region between indices 100 and 200 – where the model is expected to predict Mitterrand – there are indeed probabilities above 0.5. However, because the model frequently alternates between both classes, Gaussian smoothing is unable to sufficiently flatten these fluctuations, and the smoothed probabilities remain either unstable or completely wronged predictions.

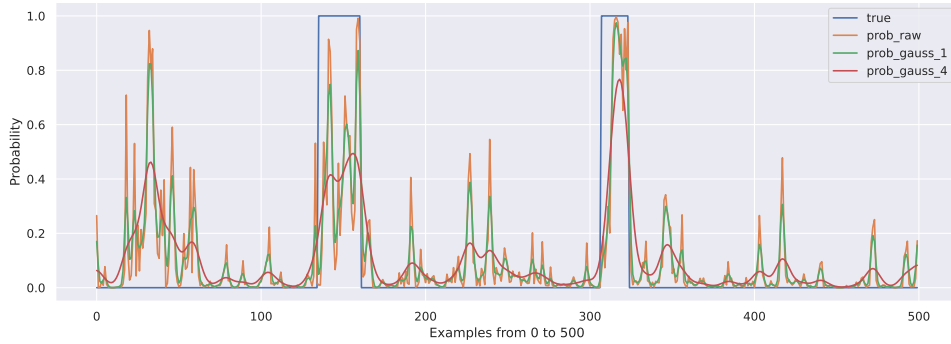


Figure 22: Gaussian smoothing applied to logistic regression predicted probabilities

As an alternative to Gaussian smoothing, we also tried a less severe **median smoothing**. Given a kernel (window) size k , for each probability we consider $\frac{k}{2}$ values before and $\frac{k}{2}$ values after it, sort these values, and take their median. Figure 23 illustrates the effect of this smoothing technique using a kernel size of 5 (i.e., 2 values before and 2 values after each point). The issue is similar to what we observed with Gaussian smoothing (with $\sigma = 1$): median smoothing can correct some Chirac predictions, but the same problem persists for Mitterrand. When the model’s sequential probabilities are unstable (as is the case for Mitterrand between indices 100 and 200), the smoothing is not sufficient to flatten these fluctuations.

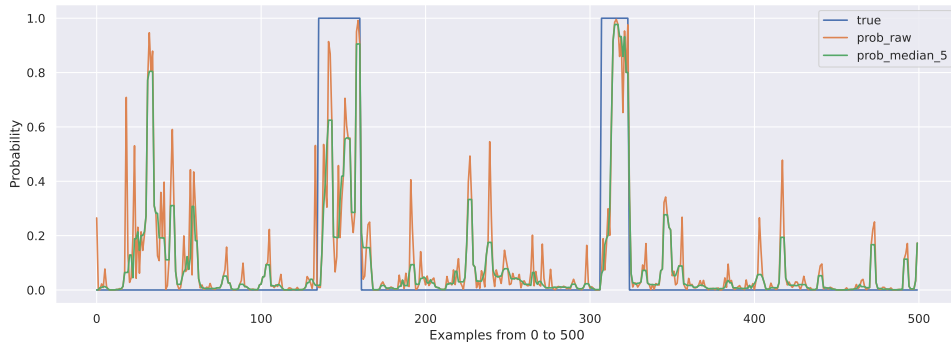


Figure 23: Median smoothing applied to logistic regression predicted probabilities

Another idea was to apply **duration smoothing** by incorporating a prior on the duration of each president’s speech, which differs significantly between presidents, as observed in Section 2.2.

The intuition is to assign a segment score $S(t, y_t)$ to each logistic regression probability $P(y_t | x_t)$, measuring how well the previous d probabilities support class y_t . Since we do not know in advance how long a speech segment lasts for a given president, we marginalize over all possible durations d for class y_t , weighting each by its prior probability. This can be written as:

$$P(y_t | x_t) = \sum_{d=3}^{D_{\max}^{y_t}} P(y_t, d | x_t) \stackrel{\text{Bayes}}{\propto} \sum_{d=3}^{D_{\max}^{y_t}} P(d | y_t) \cdot \sum_{i=\max(t-d+1, 0)}^t \log P(y_t | x_i) = S(t, y_t)$$

- y_t is the predicted class at time t
- x_t is the vectorized example at time t

To control the amount of smoothing, we combine the original probability with the segment score using a weighted sum:

$$P(y_t | x_t) = \alpha P(y_t | x_t) + (1 - \alpha) S(t, y_t)$$

where $\alpha \in [0, 1]$. When α is close to 0, more weight is given to the smoothing term, while values closer to 1 rely more on the original probabilities.

- The prior $P(d | y)$ is estimated using KDE³ with bandwidth = 1 on the training dataset.
- $D_{\max}^y$ is the maximum observed duration for class y in the training sequences.

Figure 24 shows the effect of duration smoothing for different values of α (denoted as `duration_alpha` in the legend). Unfortunately, this approach was not successful. For small values of α , the smoothing is too weak, while for larger values, it becomes too aggressive and distorts the predictions completely. In the case of Mitterrand’s sequences, probabilities do not increase quickly enough, and since his sequences are relatively short, the smoothing does not have enough time to accumulate before it is already too late and the model switches to predicting Chirac. We also believe that a limitation of this approach is that it only considers past predictions and ignores future ones. This was the main reason why we later decided to move to a bidirectional LSTM (BiLSTM) for smoothing instead of relying on a simple forward LSTM.



Figure 24: Duration smoothing applied to logistic regression predicted probabilities

Table 5 compares logistic regression performance across different smoothing techniques. Figure 25 presents the ROC and PR curves for selected variants; `no_fe_2_to_1` denotes logistic regression without FE, with Mitterrand penalized twice as much as Chirac. According to these curves, the best-performing model (the curve closest to the upper corner) is logistic regression with FE and Gaussian smoothing ($\sigma = 4$)

Model	Smoothing	Accuracy	Precision	Recall	F1-score	ROC AUC	PR AUC
Logistic regression, FE	—	0.91	0.54	0.54	0.54	0.89	0.57
Logistic regression, FE	Gauss, sigma=1	0.92	0.66	0.52	0.58	0.93	0.64
Logistic regression, FE (2)	Gauss, sigma=4	0.94	0.84	0.45	0.58	0.95	0.73
Logistic regression, FE	Median, kernel=5	0.93	0.68	0.52	0.59	0.93	0.66
Logistic regression, FE	Duration, alpha=0.5	0.91	0.68	0.13	0.22	0.86	0.46

Table 5: Logistic regression with and without smoothing performance (train/test split)

Note: `submission 1` was mistakenly submitted as Logistic Regression with FE, Gaussian smoothing ($\sigma = 4$), and `class_weight=balanced`.

³kernel Density Estimation

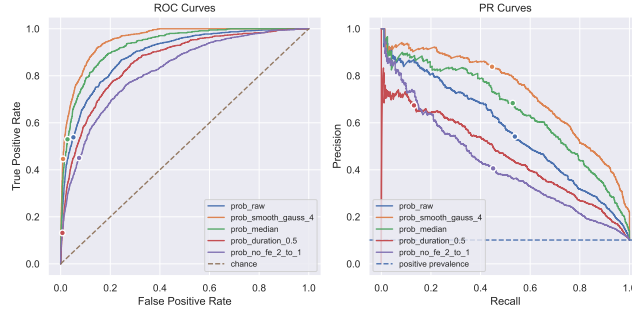


Figure 25: ROC and PR curves for selected logistic regression variants (train/test split)

6.2.3 Fine-tuning & Smoothing with BiLSTM

For finetuning BERT and training BiLSTM, the chosen weighted function is the one commonly used

$$w_c = \frac{N}{K \cdot n_c}$$

- w_c is the weight of class c
- N is the number of examples in train dataset
- n_c is the number of examples of class c
- K is the number of classes

Note : due to lack of time, we don't perform train/test split evaluation since finetuning CamemBERT on 50 000 examples takes several hours and we wanted to make a submission for the leaderboard for this model (submission 3) .

CamemBERT model was more or less confident in its predictions, but was unaware of sequential dependency of phrases. Hence, we decided to train a small BiLSTM neural net so that would correct the predictions of our finetuned CamemBERT.

Afterwards, our goal was not to rely on smothing techniques seen previously like median or gauss convolution, but instead to train BiLSTM, which as input would receive window of probabilities calculated by CamemBERT and would output smoothed probability (as labels we use 1s and 0s), which can be formalized as

$$p_{smoothed} = f(p_{t-3}, p_{t-2}, p_{t-1}, p_t, p_{t+1}, p_{t+2}, p_{t+3})$$

where f is the operation performed by BiLSTM

For edge cases, probabilities are padded with the first and last probabilities predicted by BERT, respectively. Figure 26 compares raw probabilities from fine-tuned CamemBERT with BiLSTM-smoothed probabilities. The most pronounced effects appear around index 300, where BiLSTM suppresses a likely spurious CamemBERT spike, and between indices 450 and 500, where BERT fluctuations are substantially flattened.

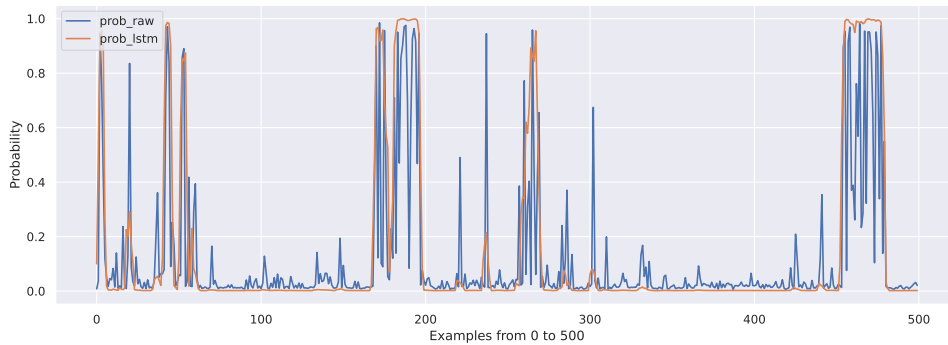


Figure 26: Smoothed probabilities with BiLSTM vs BERT model probabilities (true test dataset)

6.3 BiLSTM on speech-level

Initially, we did not realize that `speech_id` would be available in the real test set, so we did not use this information in earlier models. Here, we treat each phrase as part of a longer speech. In most speeches, the same president speaks for several consecutive sentences, with possible switches between Chirac and Mitterrand. We therefore need a model that can capture both speech-level priors and sentence-level speaker cues, which naturally implies a BiLSTM. Instead of using it as a probability smoother, we feed CamemBERT-base embeddings⁴ into a BiLSTM trained at the speech level: whenever `speech_id` changes, the hidden state is reset, allowing the model to learn sequential dependencies within each speech only.

6.3.1 Span Boundary Evaluation (Lag)

After training the BiLSTM, we observed from the predicted probabilities that transitions into and out of Mitterrand segments were poorly aligned with the ground truth. We therefore define *start lag* and *end lag*: start lag measures how early or late the model enters a Mitterrand segment, while end lag measures how early or late it exits. Figure 27 illustrates how these metrics are defined.

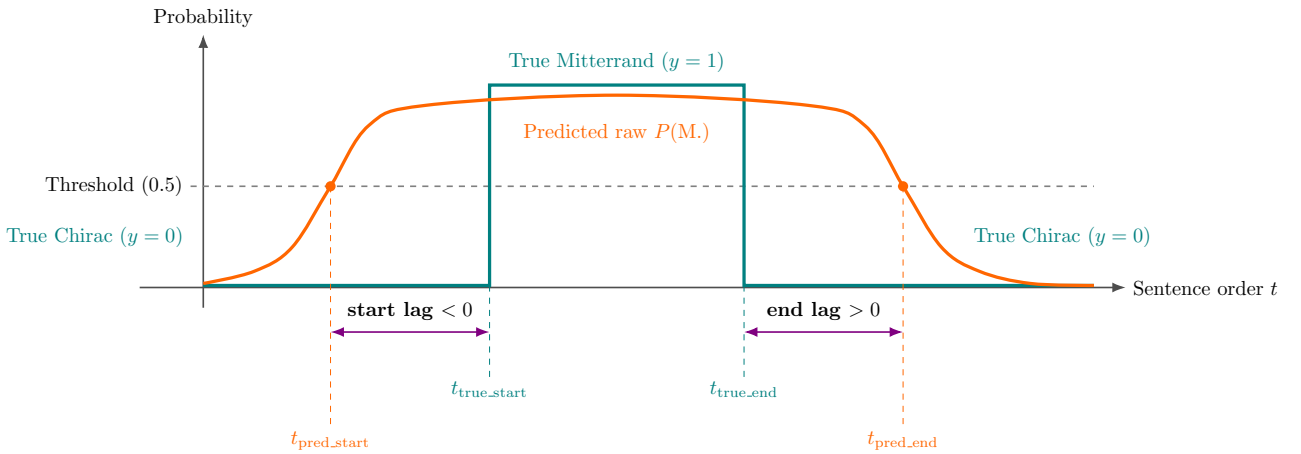


Figure 27: Visual explanation of *start lag* and *end lag*. In this case, the model enters Mitterrand earlier than it should (negative start lag) and leaves later than it should (positive end lag).

With a raw threshold of 0.5, the initially trained BiLSTM entered Mitterrand intervals too early (mean start lag: -2.30) and exited too late (mean end lag: 3.33). In practice, this corresponds to overpredicting Mitterrand by roughly 2 phrases before a speaker switch and about 3 phrases after it. Therefore, relying solely on the default threshold of 0.5 is insufficient, and the decision boundary must be increased.

To address this, we evaluated OOF⁵ predictions across varying decision thresholds and minority-class scaling configurations.

6.3.2 Training

To identify the optimal parameters, we performed a grid search over the following space:

Hyperparameter	Tested values
<code>hidden_dim</code>	{128, 256}
<code>projection_dim</code>	{256, 512}
<code>dropout</code>	{0.15, 0.2, 0.3}
<code>epochs</code>	{12, 16, 20}
<code>minority_weight_scale</code>	{1.0, 0.9, 0.8, 0.7}

Table 6: BiLSTM hyperparameter search space used during model selection

To avoid evaluating the full space of 144 combinations, we subsampled 12 configurations and evaluated them with 5-fold cross-validation grouped by speech.

⁴Tasks were split between us; since we tested both CamemBERT-base and CamemBERT-large, we did not use the same CamemBERT variant.

⁵Out-of-Fold

For each configuration:

1. Train the BiLSTM and evaluate each epoch on a validation split while tuning the decision threshold (0.80 to 0.95, step = 0.005).
2. Select the best epoch, which defines the fixed model for that configuration.

This process yields 12 fixed models. For each model, we run grouped 5-fold cross-validation, collect OOF predictions, and tune the final decision threshold (0.05 to 0.95, step = 0.005). We then evaluate each model on its OOF predictions and select the best one based on F1 score. The best-performing model parameters are reported in Appendix C.

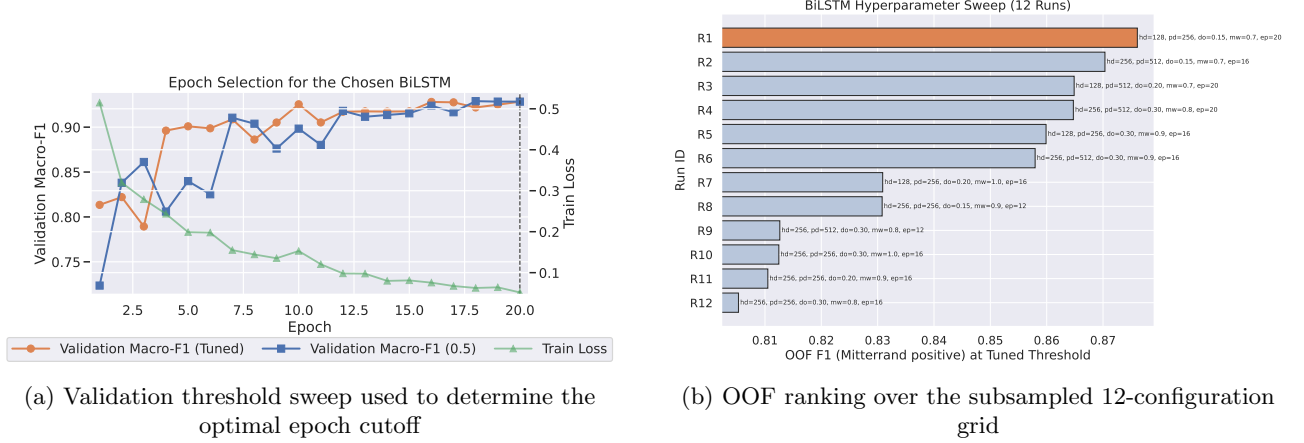


Figure 28: Hyperparameter-selection workflow for the simple speech-level BiLSTM.

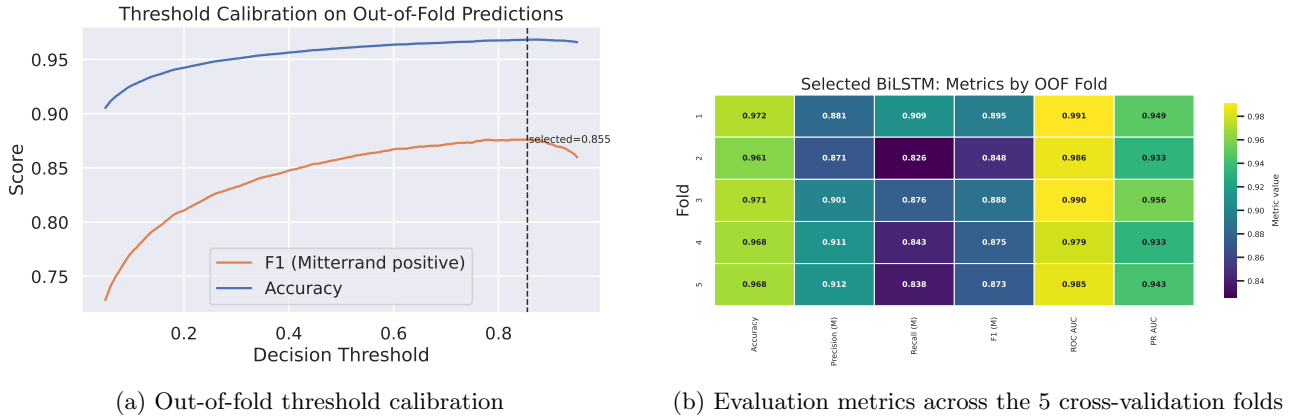


Figure 29: Decision boundary calibration and model stability diagnostics.

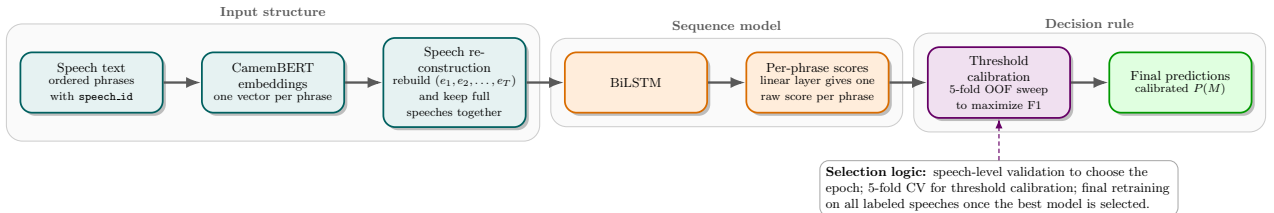


Figure 30: Simple speech-level BiLSTM pipeline for the presidents task

We found that slightly reducing the minority-class penalty (setting `minority_weight_scale` to 0.90, i.e., using 90% of the initial Mitterrand class weight) and enforcing a stricter decision boundary (predicting Mitterrand only when $P(M) \geq 0.895$) pushed weak transition probabilities back toward the Chirac class. As a result, the mean start lag decreased to -0.39 and the mean end lag to 0.41 .

6.4 Model Comparison and Ensembling

Our final model – which secured first place on the leaderboard – is an ensemble of two deep-learning approaches: a fine-tuned CamemBERT-base classifier, which captures rich local semantic information, and the sequence-aware BiLSTM introduced in the previous section. We use a simple weighted logit fusion, where the final prediction is obtained by interpolating the logits of the 2 base models with a mixing coefficient α selected from grouped OOF predictions.

Note: This time BERT was fine-tuned by appending the previous and next phrases from the same speech, yielding the input format `phrase(t-2) [SEP] phrase(t-1) [SEP] phrase(t) [SEP] phrase(t+1) [SEP] phrase(t+2)`

$$p_{\text{fusion}} = \sigma(\alpha \text{logit}(p_{\text{BiLSTM}}) + (1 - \alpha) \text{logit}(p_{\text{CamemBERT}})),$$

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

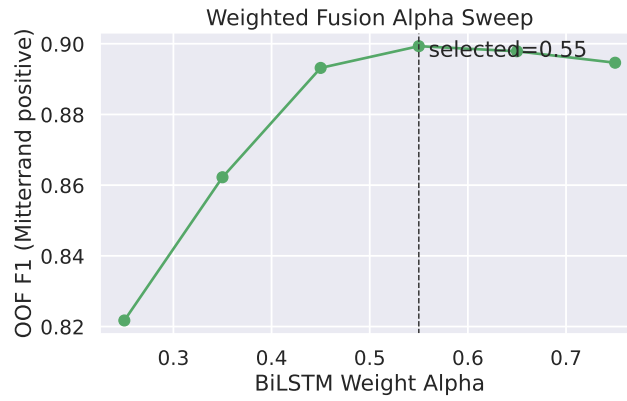


Figure 31: Parameter sweep for the fusion interpolation weight (α), identifying the optimal balance to maximize out-of-fold F1.

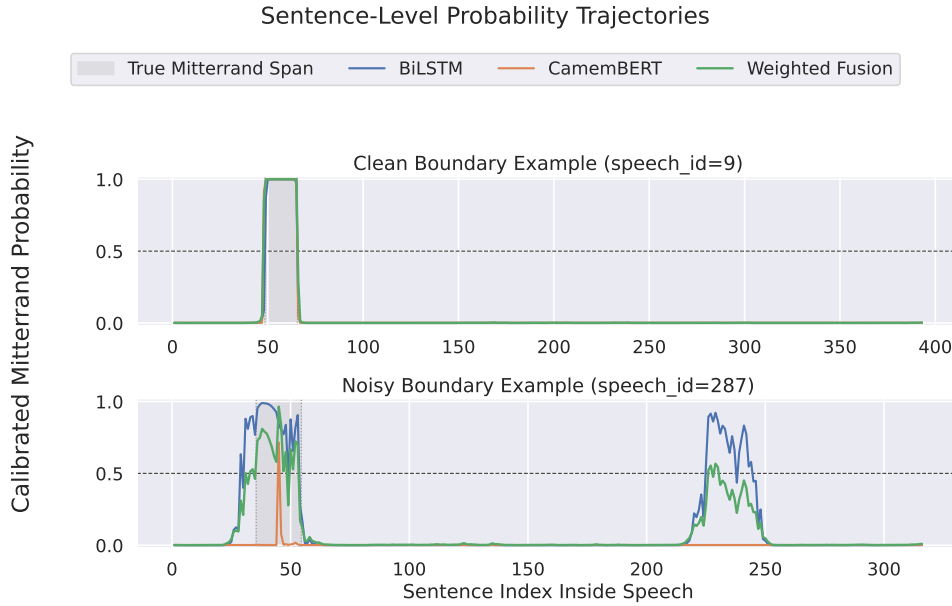


Figure 32: Probability trajectories of the calibrated simple BiLSTM

Table 7 summarizes our progress toward achieving first place on the leaderboard for the presidents dataset:

Model	Variant	Accuracy	Precision	Recall	F1	ROC AUC	PR AUC
BiLSTM (7)	Sweep winne	0.964	0.863	0.867	0.865	0.984	0.934
CamemBERT-base	Finetune	0.954	0.972	0.670	0.793	0.960	0.913
Ensembling (8)	Weighted logit, alpha=0.55	0.974	0.917	0.883	0.899	0.989	0.958

Table 7: Grouped out-of-fold comparison across the main report-safe models.

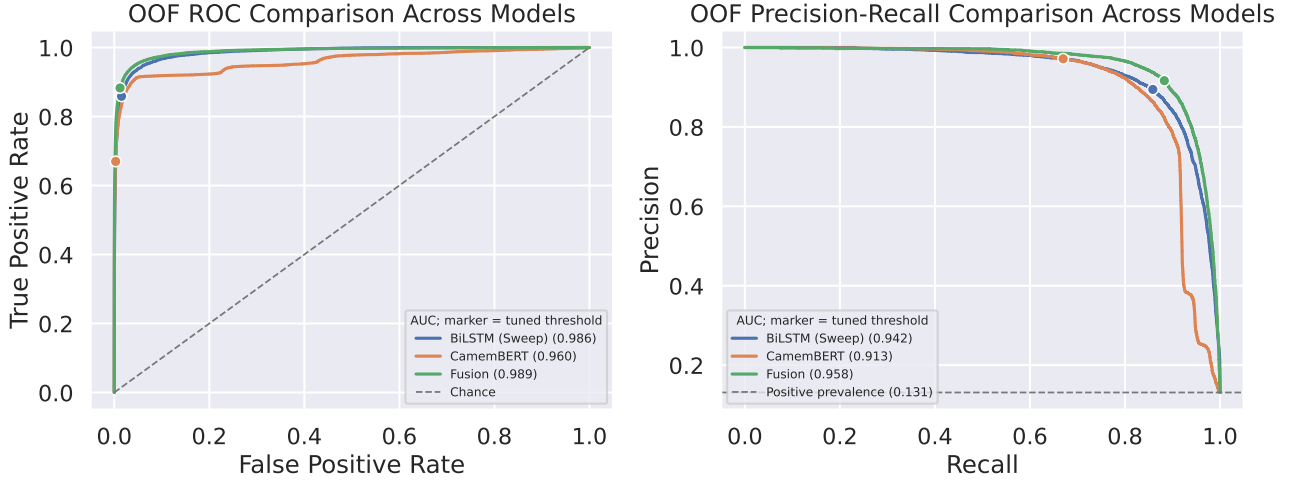


Figure 33: ROC and PR Curves for deep models

7 Conclusion

In this project, we evaluated a wide range of vectorization techniques, from simple representations to state-of-the-art transformer embeddings. The movies dataset corresponds to a standard binary classification setting, where fine-tuned BERT models achieved the best performance. The presidents dataset, however, was substantially more challenging because of its sequential structure. To reach the top leaderboard score, we explored several approaches: feature engineering to make logistic regression sequence-aware, classical smoothing methods (e.g., Gaussian and median), and more advanced strategies such as duration priors and BiLSTM-based smoothing. Ultimately, the strongest results came from training a speech-level BiLSTM (rather than phrase-level models) and combining it with a fine-tuned CamemBERT model.

Appendix

A Finetuning HuggingFace transformers

Hyperparameter	CamemBERT -large	BERT-base uncased	RoBERTa -base	CamemBERT -base
Learning rate	5e-6	5e-6	5e-6	5e-6
Batch size	16	8	8	8 (train), 16 (eval)
Training epochs	3 (all layers)	3 (all layers)	1 (classifier head) + 5 (all layers)	4 (all layers)
Warmup ratio	0.1	0.1	0.1	0.1
Weight decay ⁶	0.01	0.01	0.01	0.01
Max grad norm	1	1	1	1
Dropout	0.1	0.2	0.2	0.1

Table 8: Fine-tuning hyperparameters used for pretrained transformer models.

B BiLSTM for probability smoothing

Parameter	Value
Window size	7
Context size (k)	3
Hidden dimension	64
Number of layers	2
Dropout	0.2
Number of epochs	30
Batch size	128
Learning rate	1e-4

Table 9: Hyperparameters used for training the BiLSTM smoother

⁶Weight decay is not applied to bias parameters and LayerNorm weights, since these parameters primarily serve scaling and shifting functions rather than learning feature representations.

C Speech-level BiLSTM

Hyperparameter	Value	Description
hidden_dim	128	LSTM memory capacity
projection_dim	256	Initial input flattening size
num_layers	1	Single bidirectional sequence
dropout	0.15	Prevents dense overfitting
batch_size	8	Handled per complete speech
epochs	20	Max training duration
learning_rate	1×10^{-3}	Optimizer shift size
weight_decay	1×10^{-2}	L2 regularization norm
gradient_clip	1.0	Guards against exploding norms
minority_weight_scale	0.70	Dampens false Mitterrand alerts
validation_size	0.15	Hold-out ratio measuring epoch fit
n_folds	5	OOE evaluation clusters
seed	42	Reproducibility baseline

Table 10: Frozen hyperparameters for the best single BiLSTM